Guia rápido: problemas de Git e GitHub desta conversa

1. Erro ao dar `git push`: *"Updates were rejected because the remote contains work that you do not have locally"*

Situação
Você tentou fazer:

```
git push -u origin main
```

e recebeu:

- `! [rejected] main -> main (fetch first)`
- `Updates were rejected because the remote contains work that you do not have locally`

Motivo
O branch `main` do **GitHub** já tinha commits que não existiam no seu repositório local. O Git bloqueia o push para não sobrescrever o que está lá.

Como resolver (fluxo que você usou)

1. Baixar o que já estava no remoto, aceitando históricos diferentes:

   ```
   git pull origin main --allow-unrelated-histories
   ```

2. Resolver conflitos se aparecerem, depois:

   ```
   git add .
   git commit -m "Merge histories"
   ```

3. Enviar de volta:

   ```
   git push -u origin main
   ```

Depois disso, o `main` local e remoto ficaram sincronizados.

2. “Como apagar arquivos específicos do repositório (no GitHub)”

Situação

Havia arquivos indesejados (`img/`, `app.js`, `index.html`, `style.css`) já versionados e aparecendo no GitHub.

Como remover do repositório

Na pasta do projeto:

1. Remover os arquivos do versionamento:

   ```
   git rm -r img      # se quiser apagar a pasta inteira
   git rm app.js index.html style.css
   ```

   (ou apagar pelo explorador e, em seguida, usar `git rm -r` para registrar a remoção)

2. Registrar e enviar:

   ```
   git commit -m "Remove arquivos antigos"
   git push origin main
   ```

Os arquivos somem da listagem do GitHub (continuam só no histórico de commits).

3. “Untracked files” no `git status`

Situação

O `git status` mostrou algo como:

```
Untracked files:
  "djkjasdlk\30ß\247fjlsakd"
```

Significado

É um arquivo que existe na pasta, mas **não está sendo rastreado** pelo Git (nunca recebeu `git add`). Ele não vai para o GitHub a menos que você faça `git add` e `git commit`.

Como remover

- Se não precisa dele, basta apagar o arquivo pelo explorador/VS Code.
- Ou, para remover todos os arquivos não rastreados de uma vez:

  ```
  git clean -f
  ```

(use com cuidado; apaga todos os untracked).

4. Desfazer `git add` de vários arquivos (tirar do "Changes to be committed")

Situação
Você tinha rodado `git add` em um monte de arquivos/pastas (aparecendo como `new file` em verde).

Como limpar o stage sem apagar arquivos

```
git reset
```

Isso tira todos os arquivos do *stage* (voltam para o estado "untracked" ou "modified", mas continuam no disco).

Se quiser também apagar os arquivos/pastas em si (por exemplo uma pasta inteira criada por engano):

```
git rm -r nome_da_pasta
```

ou apagar manualmente no explorador.

5. Voltar para o estado do GitHub (jogar fora mudanças locais)

Situação
Você queria "deixar tudo igual ao GitHub", descartando o que estava diferente na máquina local.

Comando

```
git fetch origin
git reset --hard origin/main
```

- `fetch` traz o estado mais recente do remoto.
- `reset --hard origin/main` faz o seu branch local `main` ficar **idêntico** ao `origin/main`, apagando mudanças locais (commits locais que não estão no remoto e modificações no working directory).

6. Apagar o último commit (quando ainda não deu `push`)

Cenário geral

- Voltar 1 commit **mantendo os arquivos no disco** (só tira do histórico):

  ```
  git reset --soft HEAD~1
  ```

- Voltar 1 commit **apagando também as alterações dos arquivos**:

  ```
  git reset --hard HEAD~1
  ```

Se já tiver dado `git push`, depois do `reset` precisa sobrescrever o remoto:

```
git push origin main --force
```

(use com cuidado se outras pessoas usam o repositório).

7. Apagar um commit que é o único no histórico local

Na captura, o `git log --oneline` mostrava só:

```
ff9805a (HEAD -> main) backup
```

Ou seja, esse é o único commit do branch local.

7.1. Manter o que está no GitHub e jogar fora esse commit local

```
git fetch origin
git reset --hard origin/main
```

Isso faz seu `main` local voltar a ser exatamente o `main` do GitHub.

7.2. Zerar tudo (inclusive no GitHub) e começar do zero

Se você quiser apagar esse commit **também do remoto**, pode recriar o branch:

```
git checkout --orphan temp
git rm -rf .
git commit --allow-empty -m "Initial empty commit"
git branch -M main
git push origin main --force
```

- `--orphan` cria um branch sem histórico.
- `git rm -rf .` limpa todos os arquivos rastreados.
- `--force` sobrescreve o branch `main` remoto com esse novo histórico vazio.

Se quiser, posso transformar este resumo em um documento completo no formato Word/PDF para você baixar, com seções numeradas e exemplos mais detalhados de cada comando.